PYTHON ACTIVITY-BASED LEARNING

~PEDDADA TEJASWINI
~CSM
~26001232

PROBLEM:

PROBLEM STATEMENT:

University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules

OBJECTIVE:

To develop a program that evaluates scholarship applicants based on aggregate marks, subject-wise minimum marks, attendance percentage, and family income. The system identifies eligible students, ranks them according to merit, and resolves ties using predefined rules.

PROGRAM:

```
# University Scholarship Allocationstudents = [ ["Ravi", 90, 75, 85, 40000], ["Sita", 90, 80, 90, 35000], ["Ram", 65, 70, 80, 30000], ["Priya", 85, 60, 75, 45000]]eligible = []for s in students: name, marks, subject, attendance, income = sif (marks >= 70 and subject >= 50 and attendance >= 75 and income <= 50000): eligible.append(s)# Ranking: aggregate marks, subject marks,# attendance (higher values first), income (lower first)eligible.sort(key=lambda s: (-s[1], -s[2], -s[3], s[4]))print("Eligible Scholarship Applicants:")print("Name Marks Subject Attendance Income")for s in eligible: print(s[0], s[1], s[2], s[3], s[4])print("\nScholarship Ranking:")for rank, s in enumerate(eligible, start=1): print(rank, "-", s[0])
```

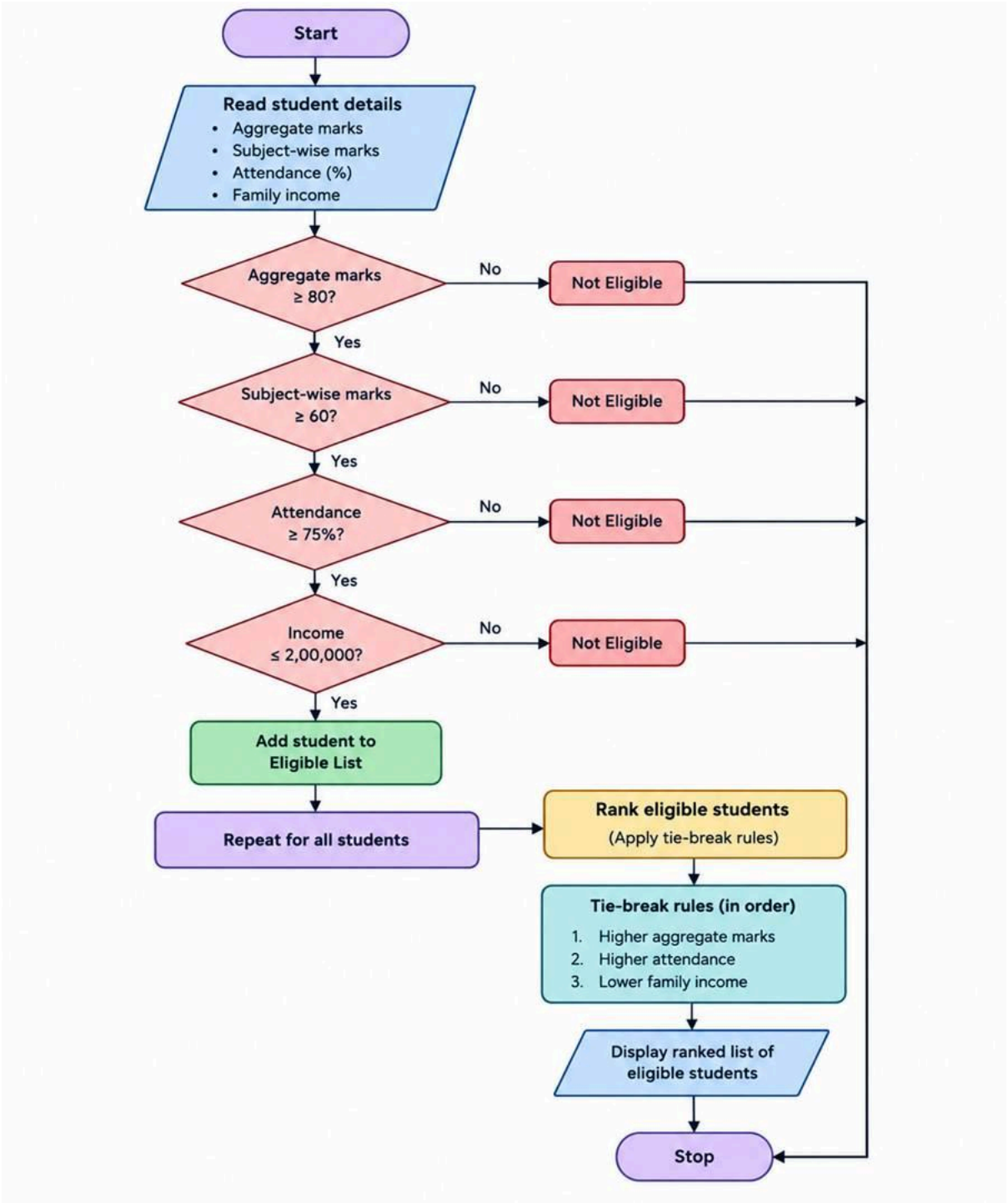
ALGORITHM:

1. Start.
2. Read the applicant details: name, aggregate marks, subject marks, attendance, and family income.
3. Check whether aggregate marks are at least 70.
4. Check whether subject marks are at least 50.
5. Check whether attendance is at least 75%.
6. Check whether family income is ₹50,000 or less.
7. If all conditions are satisfied, add the applicant to the eligible list.
8. Sort eligible applicants by:
    - Higher aggregate marks.

- Higher subject marks.
- Higher attendance.
- Lower family income.

9. Assign ranks to the eligible applicants.
10. Display the eligible list and scholarship ranking.
11. Stop.

FLOWCHART:



PSEUDOCODE:

```
BEGIN
        INPUT student details
        eligible_list = empty list

        FOR each student
        IF aggregate_marks >= 70 AND
       subject_marks >= 50 AND
        attendance >= 75 AND
       family_income <= 50000 THEN

        ADD student to eligible_list
        END IF
        END FOR

        SORT eligible_list BY
     aggregate_marks DESCENDING,
        subject_marks DESCENDING,
        attendance DESCENDING,
        family_income ASCENDING

        rank = 1

        FOR each student in eligible_list
        DISPLAY rank, student details
        rank = rank + 1
        END FOR
END
```

OUTPUT:

Eligible Scholarship Applicants:

Name	Marks	Subject	Attendance	Income
Sita	90	80	90	35000
Ravi	90	75	85	40000
Priya	85	60	75	45000

Scholarship Ranking:

1 - Sita

2 - Ravi

3 - Priya